ECOTRAIN

Making Energy Prediction Smarter

PART A
Energy Consumption Prediction

PART B
Comparing Four Optimizers

A project inspired by learning how optimizers work and turning that lesson into a practical energy prediction app.

Project Report | Pink Edition

1. About the Project

EcoTrain is a small project I developed while learning about different optimizers. I wanted to go beyond reading about them and see how they perform in practice. That led me to build a simple app around energy consumption, with one part for making predictions and another for comparing four optimizers.

What I wanted to achieve

- Create a simple page where users can enter readings and get an energy prediction.
- Try four optimizers on the same regression task and compare their results.
- Look at training time and prediction error to understand the differences.
- Make the project easy to explore through a Streamlit interface.

2. Part A: Energy Consumption Prediction

The first part estimates active power from the values entered by the user. These include reactive power, voltage, intensity, and readings from three submeters. Before the values reach the trained model, the app uses a saved scaler to prepare them in the same way as the model expects.

How it works

- Enter the requested electrical readings.
- The app prepares the values using the saved scaler.
- The trained model processes the readings.
- The predicted active power appears on the page.

A note on accuracy: The quality of a prediction depends on the data used to train the model, the correct order of input features, and the matching model and scaler files.

3. Part B: Comparing Four Optimizers

The second part looks at how four optimizers train the same type of neural network on a regression dataset. Each run records how long training takes and how far the predictions are from the test values. The results can change with the learning rate, data, and random starting point, so the comparison should be based on the actual runs rather than assuming one method is always the best.

Optimizer	In simple words
SGD	A basic method that updates model weights step by step.
Momentum	Uses the direction of earlier updates to make learning smoother.
RMSprop	Adjusts update sizes based on recent gradient values.
Adam	Combines adaptive step sizes with information from earlier updates.

What the measurements mean

- **Training time:** How long the model takes to train.
- **Test MSE:** Measures larger errors more heavily. A lower value is generally better.
- **Test MAE:** The average size of the prediction errors. A lower value is generally better.

4. Other Experiments

The app also tests whether reducing the learning rate when progress slows can help training. It includes an early-stopping option, which can stop a run when validation results no longer improve. Repeating experiments with different random seeds makes it easier to see whether the results stay similar.

5. Tools Used

Tool	How it is used
Python	The main language used to build the project.
Streamlit	Creates the interactive web app.
TensorFlow and Keras	Build and train the neural network and make predictions.
NumPy and Pandas	Help work with numbers and data.
Scikit-learn	Prepares datasets, splits data, and scales input values.
Matplotlib	Displays experiment charts.
Joblib	Loads the saved scaler.

6. Things I Learned

- A learning rate that works for one optimizer may not work as well for another.
- Training results should be checked for invalid values such as NaN or infinity.
- A fair comparison means keeping the dataset, model structure, and evaluation method consistent.
- Running tests more than once can give a clearer picture of how reliable the results are.

7. Conclusion

EcoTrain brings together two things I wanted to explore: predicting energy consumption and understanding how optimizers behave. The first part gives users a prediction from electrical readings. The second part lets them compare SGD, Momentum, RMSprop, and Adam using training time and test errors. For me, the project was a way to turn something I was learning into a working application.

Ideas for the next version

- Explain the source and units of the energy data more clearly.
- Add easy-to-read charts for training progress and optimizer results.
- Allow users to export experiment results.
- Test the prediction model on more representative energy data.